

Ghost Channel SDK 开发者使用手册

Developer Guide for ghost-channel-sdk

QCM Research & Engineering Team

Quantum Collaborative Model (QCM) Initiative

qcm@local

Submission Draft

Abstract

1. 手册定位

这份手册不是单纯的“安装说明”，而是`ghost-channel-sdk`的完整使用入口。它要同时解决三类问题：

1. **工程师问题**：如何理解对象、接口、测试与目录结构，并继续实现协议能力。
2. **集成者问题**：如何把 SDK 放进现有系统，快速跑通演示与验证。
3. **AI 问题**：如何让模型把这套 SDK 当作一组可执行规则来使用，而不是一堆零散代码。

1.1 读完这份手册后，你应该能做到

读者	读完后应能做到
协议工程师	看懂协议对象、Schema、ACK 生命周期、Replay 规则
Python 开发者	在本地跑通 Memory / Workflow Demo，理解对象流与恢复逻辑
TypeScript 开发者	看懂 TS SDK 的当前能力与缺口，继续补齐实现
集成者	判断从哪里接入、如何校验、如何跑最小 PoC
AI 模型	按协议对象、执行步骤、错误码与约束进行动作推理

2. 五分钟快速开始

2.1 如果你是 Python 开发者

```
cd ghost-channel-sdk/python
pip install -e .
python -m ghost_channel.cli validate-assets
python -m ghost_channel.cli sync-memory-demo
python -m ghost_channel.cli workflow-demo
```

2.2 如果你是 TypeScript 开发者

```
cd ghost-channel-sdk/typescript
npm install
node --test test/sdk.test.js
```

2.3 如果你是 AI 或自动化代理

优先读取：

1. `schemas/`
2. `examples/`
3. `example-schema-map.json`
4. `python/ghost\_channel/sdk.py`
5. `typescript/src/index.ts`

然后遵循：

- 先验证资产 - 再理解对象 - 再执行同步

3. 项目结构总览

```
ghost-channel-sdk/
├── README.md
├── DEVELOPER_GUIDE_v1.0.md
├── DEVELOPER_GUIDE_v1.1.md
├── schema-registry.md
├── object-alignment-matrix.md
├── .pre-commit-config.yaml
├── .github/workflows/python-sdk-ci.yml
├── schemas/
├── examples/
├── python/
└── typescript/
```

3.1 人类说明

这是一个典型的“协议资产 + 双语言 SDK + 校验链”结构：

- `schemas/`：正式对象约束 - `examples/`：真实夹具（fixture） - `python/`：当前最成熟实现 - `typescript/`：语义对齐中的另一语言实现

3.2 工程说明

开发顺序建议：

1. 先看 `schema-registry.md` 2. 再看 `examples/example-schema-map.json` 3. 再实现 / 修改 `sdk.py` 或 `index.ts` 4. 最后跑 validator 和 tests

3.3 AI 执行说明

AI 若要操作该仓库，应遵循：

schemas → examples → mapping → sdk → tests

不得跳过 schema / example 层直接改 SDK 逻辑。

4. 核心对象总览

对象	作用	所在层
`DeltaPayload`	状态差分	协议对象
`EncryptedStream`	线协议主消息	线协议对象
`AckMessage`	回执确认	线协议对象
`SyncResult`	SDK 同步返回结果	SDK 边界对象
`ErrorObject`	标准错误载体	通用对象
`VectorClock`	因果时钟	协议对象
`AuditEntry`	审计记录	审计对象
`WorkflowStep`	工作流步骤	工作流对象
`SnapshotRecord`	快照记录	恢复对象

4.1 人类说明

记住一个原则：

- **SDK 内部对象** 用于表达和执行逻辑 - **线协议对象** 用于传输和验证

例如：- `DeltaPayload`：内部对象 - `EncryptedStream`：线协议对象

4.2 工程说明

当前已导出的 schema 文件：

```
schemas/
■■■■ delta-payload.schema.json
■■■■ encrypted-stream.schema.json
■■■■ ack-message.schema.json
■■■■ sync-result.schema.json
■■■■ error-object.schema.json
■■■■ vector-clock.schema.json
■■■■ audit-entry.schema.json
■■■■ workflow-step.schema.json
■■■■ snapshot-record.schema.json
```

4.3 AI 执行说明

AI 若要执行同步：

1. 先构造 `DeltaPayload` 2. 再封装成 `EncryptedStream` 3. 等待 `AckMessage` 4. 返回 `SyncResult`

5. Python SDK 使用

5.1 当前成熟度

Python 是当前最完整实现。

已具备：

- 真实 AES-256-GCM - `DeltaPayload` 对象流 - 字段级变化追踪 - `listAppends` 优化 - `EncryptedStream` 对象流 - ACK 生命周期 / completionMode - replay window - `SnapshotRecord` 快照链恢复 - schema 校验接入主流程 - CLI / demo runner

5.2 基本构造

```
from ghost_channel import GhostChannelSDK, GhostChannelConfig

sdk = GhostChannelSDK(
    GhostChannelConfig(
        compression_level=9,
        semantic_threshold=0.70,
        audit_enabled=True,
        max_retry=3,
        completion_mode="apply",
        await_ack=False,
        ack_timeout_ms=500,
        replay_window_size=1024,
    )
)
```

5.3 Memory Sync

```
result = await sdk.sync_memory_delta(
    source_role="secretary_v1",
    target_role="researcher_v1",
    old_state={"__version__": "v1", "x": 1},
    new_state={"__version__": "v2", "x": 2, "y": 3},
    semantic_filter="protocol scope",
)
```

可读取：

```
sdk.last_delta_payload
sdk.last_encrypted_stream
sdk.last_validation_report
sdk.get_audit_trail()
sdk.get_stats()
```

5.4 Workflow Sync

```
result = await sdk.sync_workflow_state(
    workflow_id="wf_demo",
    step_id="step_01",
    step_state={"status": "completed", "payload": {"x": 1}},
    dependencies=[],
)
```

5.5 ACK 处理

```
await sdk.receive_ack(AckMessage(...))
```

当前支持：

- schema 校验 - ACK 语义校验 - monotonic progression - replay / duplicate 控制 - `completion\_mode` 终态判断

5.6 恢复

```
recovered = await sdk.recover_from_failure(
    "step_01",
    {"status": "fallback"},
)
```

恢复逻辑：

- 优先从 `SnapshotRecord` 链中选取最新可恢复快照 - 否则 fallback 到 `last\_known\_state`

6. TypeScript SDK 使用

6.1 当前成熟度

TypeScript 已进入协议语义 MVP 阶段，但总体仍弱于 Python。

已具备：

- `DeltaPayload` 对象流 - `EncryptedStream` 对象流 - `lastDeltaPayload` - `lastWorkflowDeltaPayload` -
 `lastEncryptedStream` - ACK 语义 - completionMode / awaitAck - replay / duplicate 基础控制 - Snapshot 恢复 -
 validateAssets

6.2 基本构造

```
import { GhostChannelClient } from './src/index.ts'

const client = new GhostChannelClient({
  compressionLevel: 9,
  semanticThreshold: 0.7,
  auditEnabled: true,
  maxRetry: 3,
  completionMode: 'apply',
  awaitAck: false,
  ackTimeoutMs: 500,
  replayWindowSize: 1024,
})
```

6.3 Memory Sync

```
const result = await client.syncMemoryDelta(
  'secretary_v1',
  'researcher_v1',
  { __version__: 'v1', x: 1 },
  { __version__: 'v2', x: 2, y: 3 },
  'protocol scope',
)
```

6.4 Workflow Sync

```
const result = await client.syncWorkflowState(
  'wf_demo',
  'step_01',
  { status: 'completed', payload: { x: 1 } },
  [],
)
```

6.5 ACK / awaitAck

当：

```
awaitAck: true
```

时，TypeScript 版也会等待 ACK 终态。

7. Schema / Example / Mapping 资产链

7.1 Schema

`schemas/` 是协议对象的正式约束层。

7.2 Example

`examples/` 是真实协议对象夹具。

7.3 Mapping

`examples/example-schema-map.json` 明确了：

```
example → schema
```

7.4 Registry

`schema-registry.md` 给出完整 schema 清单与职责。

7.5 Alignment Matrix

`object-alignment-matrix.md` 对齐：

- RFC - 规范稿 - JSON Schema - Python SDK - TypeScript SDK

8. 校验与质量保障

8.1 Python

```
python -m unittest python/tests/test_sdk.py
python -m unittest python/tests/test_schema_validator.py
python -m unittest python/tests/test_cli.py
```

8.2 TypeScript

```
node --test typescript/test/sdk.test.js
```

8.3 pre-commit

```
pre-commit run --all-files
```

8.4 CI

当前已接入：- Python GitHub Actions CI

9. CLI / Demo Runner

可用命令

```
python -m ghost_channel.cli validate-assets
python -m ghost_channel.cli sync-memory-demo
python -m ghost_channel.cli workflow-demo
```

用途

- 资产校验 - Memory 路径演示 - Workflow 路径演示 - 快速验收 SDK 是否可运行

10. 推荐开发 workflow

修改协议对象时

1. 先改 RFC / 协议规范稿 2. 再改 schema 3. 再改 example 4. 再改 SDK 类型 5. 最后改 SDK 实现与测试

修改 SDK 行为时

1. 先写测试 2. 再最小实现 3. 再跑 validator 4. 再回写文档

11. AI 直接执行指南

AI 若要使用这套 SDK / 协议资产，应遵循：

1. 先读 `schema-registry.md` 2. 再读 `examples/example-schema-map.json` 3. 再看 `schemas/` 4. 再看 `sdk.py` / `index.ts`
5. 不得跳过 schema / example 层直接修改实现

AI 执行优先路径：

schema → example → mapping → sdk → tests

12. 当前能力边界

Python SDK

- 已是协议对象驱动 + 真实加密 + 双路径 ACK 等待的 MVP

TypeScript SDK

- 已是带对象流与 ACK 语义的 MVP - 仍弱于 Python 在真实加密和 schema 主流程接入上的完整度

共同边界

- 仍属于 MVP / 原型阶段 - 不等于大规模生产环境已验证

13. 下一步建议

建议优先顺序：

1. 持续保持 Python / TypeScript 对齐 2. 把 RFC / 规范稿与 SDK 同步演进 3. 增加 conformance test suite 4. 准备开源发布结构

本手册是面向当前 MVP 阶段的完整版开发者入口。